



COURSE MATERIAL

Language Specs

CMSC 124 PLS

Prepared by: Rene Andre Bedonia Jocsing

Published: September 04, 2026

Deadline: December 09, 2026

Your group's `README.md` is the specification for the language you're building. It's the first thing anyone sees on your repository. It's what I read before your defense. By December, it's the only document that says what your language is supposed to do. This template is what goes in it.

This isn't a form to fill out once. It grows across the five laboratory activities, one section at a time. Each section becomes due in the activity that forces you to decide the question it asks. You can't write the truthiness rule in September because you haven't built an evaluator yet. You should write your keyword list in September, because if you don't, your scanner tests will encode a vocabulary nobody agreed on.

This document is the reference you'll argue from when your group disagrees about what the language does, which happens more often than you'd expect once a lot of people are editing one parser. It's also the thing your committed tests are supposed to encode. A test suite that contradicts the specification means one of the two is wrong.

How to Use This Template

Copy the block at the end of this document into your repository's `README.md` and start filling it in. Delete the bracketed prompts as you replace them. Keep the section order, since it's the order I read them in and the order the activities need them.

Fill each section in the activity that needs it, and commit that edit alongside the code it describes instead of in one documentation dump at the end. A specification commit that lands in the same hour as the feature it documents tells me something about how your group works. Twelve of them landing the night before a defense also tells me something. Only one of these is a good look.

Keep it honest. A specification that describes the language you meant to build is worse than no specification, since it sends a reader looking for bugs that don't exist and hides the ones that do. When your implementation and your specification disagree and you don't have time to fix the code, change the document and note it under known limitations.

Nothing here forbids you from adding sections. A language with pattern matching, string interpolation, or a module system needs to say so somewhere, and there's no prompt for it below.

What Each Section Is for

Creators

Full names and GitHub usernames of every member. Your individual grade comes from commits attributed to you, so this section and your commit history have to name the same people. See the syllabus on individual contribution scoring.

Overview

One paragraph covering what the language is for, who'd want to use it, and what it feels like to write. If your language is a domain-specific one for a game, a scripting language for text processing, or a general-purpose language with one unusual idea, this is where you say so. Write it in the first activity and revise it in the last, when you know whether you accomplished what you set out to do.

Host Language and Build

The language you implemented your interpreter in, the version metadata file that pins it, and the command that builds. This exists so a reader who clones your repository at midnight can get it running without asking you anything, which is, unfortunately, the position I'm in when I review your code and refactors after a defense.

Running it

The interface from the run contract, written out concretely for your project: the flag that dumps tokens, the flag that prints the parse, the flag that evaluates expressions, plain execution, and so on. State the exact spellings you chose. Also record which exit code each class of failure produces. You're the one who decided which of your errors are static and which are runtime.

Due in Lab 1 for the tokenize flag, and extended in each activity after it.

File Extension

The extension your language's source files use. It has to match the `ext` field in every `manifest.json` under `tests/`. Getting this wrong is the single fastest way to a red continuous integration run that has nothing to do with your interpreter.

Lexical Structure

The vocabulary of your language. It's the first thing your scanner encodes. All five parts are due in Lab 1.

The keyword list, with what each word does. Every word here is a word your users can't use as a variable name, so a long list has a cost.

The operator table, with each operator's category, how many operands it takes, its associativity, and its precedence relative to the others. You can write the first three columns in Lab 1. Precedence and associativity become binding in Lab 2, when your grammar has to encode them. This table is where you notice that you never decided whether `-` binds tighter than `*`.

The literals, with their syntax and what value each one produces. Say whether numbers are integers, floats, or both, whether strings support escape sequences and which ones, whether strings can span lines, and what your language calls the absence of a value.

The identifier rules: which characters may start one, which may continue one, and whether case matters. "Case-sensitive" is a decision with consequences, so note it down explicitly instead of leaving it implied.

The comment syntax, including whether comments can nest and whether a comment may appear in the middle of a line. This one has a second audience. The test harness needs your comment token to read inline annotations from Lab 3 onward, so whatever you write here goes in `comment_prefix`.

Whitespace and Termination

Whether whitespace matters to the scanner, how a statement ends, and what delimits a block and a grouping. Python and C answer these differently. Both answers are respectable. Answer them in Lab 1 even though statements don't exist yet, since the answer constrains your scanner.

Token Output Format

The exact text `--tokenize` prints for one token, with an example. This is part of your specification rather than decoration, since the harness compares your token output byte for byte against expectations you committed. Freeze it before you write your first test file, and if you change it later, say so in the changelog so a reader can work out why an old commit's expectations look different.

Due in Lab 1.

Grammar

The context-free grammar for your language, in the notation the Lab 2 manual uses. It has to be unambiguous. It has to encode your precedence and associativity choices in how the rules delegate to each other.

The grammar grows in every activity after Lab 2: statements and declarations in Lab 4, control flow and functions in Lab 5. Keep one grammar that describes the current language instead of five fragments per activity, and update it in the same commit as the parser change. A grammar that disagrees with the parser is the most common defect in this section. It's an easy one for me to find by reading two files.

Parse Output Format

How `--parse` prints a tree: the parenthesized form you chose, how a grouping appears, and how numbers are formatted. Two groups can both be right here and produce completely different text. You decide the format.

Due in Lab 2.

Semantics

What your language means, as opposed to what it looks like. This section carries most of the design decisions you'll be asked to defend.

Values and types, in Lab 3: what runtime values exist, and how you represent them in a host language that probably isn't dynamically typed, since no one chose Julia.

Value printing, in Lab 3: how a number prints, how the nil value prints, and whether strings print with quotes. Now that a harness compares strings, `5` and `5.0` are different answers to the same question.

Truthiness, in Lab 3: which values count as true in a condition. Write the whole rule, with no example standing in for it.

Operator semantics, in Lab 3: which operand types each operator accepts, what `+` does when given two strings or one of each, how equality treats mismatched types, and what division by zero does. Returning an infinity and raising a runtime error are both defensible. The specification is where you commit.

Scope and bindings, in Lab 4: whether redeclaring a name in the same scope is legal, what an uninitialized variable holds, and how shadowing behaves. Also whether reading an undefined name is caught before execution or at runtime. That choice decides whether such a program exits 65 or 70.

Control flow and functions, in Lab 5: whether your logical operators return booleans or operands, how the dangling else resolves, whether a function's closure captures a loop variable per iteration or once, and what a function with no return statement produces.

Native Functions

A table of the functions your interpreter provides that users didn't write: name, how many arguments, what it returns, and any caveat. These are part of your language's surface, so a user reading only this document should be able to call them correctly.

Due in Lab 5.

Errors and Diagnostics

The format of your error messages and the mapping from failure to exit code: which conditions are static errors, which are runtime errors, and what a user sees in each case. One example of each is worth more than a paragraph describing them.

Start it in Lab 1 with your lexical errors and extend it as new failure modes appear.

Testing Conventions

Which test folder covers which activity, what mode each folder's manifest uses, and which flag it passes. This is three lines and it saves the next person, possibly you in November, from opening five manifests to remember where the parser tests live. Even better: document what each test is supposed to prove.

Sample Code

A few short programs in your language, each with the output it produces. Early on, the samples are the only way a reader can see what your language looks like. Later, they're the honest test of whether it's pleasant to write in. Replace the early ones as the language grows. A sample that no longer runs is worse than no sample.

Design Rationale

Why the language is the way it is. This is the section I read most carefully. It's the one that separates a language from a pile of features. Not every choice needs a paragraph, but the ones that surprised you do: the syntax you borrowed and from where, the feature you cut and why, the decision you reversed halfway through Lab 4, the thing you'd do differently if you started again in January.

Vague approval of your own work is worth nothing here. "We chose semicolons because it's cleaner" says less than "we chose semicolons after our scanner tests showed that significant newlines forced us to emit token positions we hadn't planned for."

Known Limitations

What doesn't work, what you didn't implement, and where your interpreter behaves worse than you'd like. Deep recursion exhausting the host stack, unimplemented escape sequences, an error message that doesn't report a column. All of it belongs here.

Writing this section isn't an admission of failure. Skipping it isn't a way to hide anything. I'll find the limitation either way, so me finding it in your own documentation is much better for you than me finding it by surprise.

Changelog

Write one short entry per activity, covering what changed in the language itself rather than in your code. It's the cheapest way to answer the question I'll ask at every defense after the first: what changed since the last one and why?

The Template

Copy everything inside the block into your README.md.

```
1 # [Language name]
2
3 ## Creators
4
5 - [Full name] ([github-username])
6 - [Full name] ([github-username])
```

```

7
8  ## Overview
9
10 [One paragraph: what the language is for, who would use it, what writing it
11 feels like.]
12
13 ## Host language and build
14
15 - Host language: [language and version]
16 - Version metadata: [file that pins it, e.g. rust-toolchain.toml, go.mod]
17 - Build: `./build.sh`
18 - [Anything a fresh clone needs to know.]
19
20 ## Running it
21
22 | Command | What it does |
23 |---|---|
24 | `./run <file>` | [Executes a program. Available from Lab 4.] |
25 | `./run --tokenize <file>` | [Prints the token stream.] |
26 | `./run --parse <file>` | [Prints the parsed tree.] |
27 | `./run --eval <file>` | [Evaluates each expression and prints its value.] |
28 | `./run` | [Starts the REPL.] |
29
30 Exit codes: 0 [when], 65 [when], 70 [when].
31
32 ## File extension
33
34 `[.ext]` [Must match the `ext` field in every tests/lab*/manifest.json.]
35
36 ## Lexical structure
37
38 ### Keywords
39
40 | Keyword | Purpose |
41 |---|---|
42 | [word] | [what it does] |
43
44 ### Operators
45
46 | Operator | Category | Operands | Associativity | Precedence |
47 |---|---|---|---|---|
48 | [op] | [arithmetic, comparison, logical, assignment, other] | [unary or binary] | [left,
49   ↪ right, none] | [1 = loosest] |
50
51 ### Literals
52
53 | Kind | Syntax | Produces |
54 |---|---|---|
55 | [number] | [e.g. 42, 3.14] | [what runtime value] |
56 | [string] | [e.g. "hello", escapes supported] | [what runtime value] |
57 | [boolean] | [true, false] | [what runtime value] |
58 | [nil] | [spelling] | [what runtime value] |
59
60 ### Identifiers
61
62 - Start characters: [which]
63 - Continue characters: [which]
64 - Case-sensitive: [yes or no]
65 - [Reserved patterns, length limits, or other restrictions.]
66
67 ### Comments

```

```
68 - Line comments: [token]
69 - Block comments: [tokens, or "not supported"]
70 - Nesting: [supported or not]
71 - [Harness note: comment_prefix in tests/lab*/manifest.json is set to the
72   token above.]
73
74 ## Whitespace and termination
75
76 - Whitespace significant: [yes or no, and where]
77 - Statement terminator: [e.g. semicolon, newline, none]
78 - Block delimiters: [e.g. braces, indentation]
79 - Grouping delimiters: [e.g. parentheses]
80
81 ## Token output format
82
83   ```
84   [one line of real --tokenize output]
85   ```
86
87   [What each field means. Frozen as of Lab 1; changes are recorded in the
88   changelog.]
89
90 ## Grammar
91
92   ```
93   [Your complete context-free grammar, current as of the latest activity.
94   Unambiguous, with precedence and associativity encoded in rule structure.]
95   ```
96
97 ## Parse output format
98
99   ```
100   [one line of real --parse output, e.g. (+ 1.0 (* 2.0 3.0))]
101   ```
102
103 - Groupings print as: [form]
104 - Numbers print as: [form]
105
106 ## Semantics
107
108 ### Values and types
109
110 [What runtime values exist, and how they are represented in the host
111 language.]
112
113 ### Value printing
114
115 - Numbers: [e.g. 5 rather than 5.0]
116 - Nil: [spelling]
117 - Strings: [with or without quotes]
118
119 ### Truthiness
120
121 [The complete rule. Which values are false in a condition; everything else is
122 true.]
123
124 ### Operator semantics
125
126 - Arithmetic: [accepted operand types]
127 - `+` on strings: [concatenation, error, or coercion]
128 - Mixed types: [what happens]
129 - Comparison: [accepted operand types]
```

```

130 - Equality across types: [false, or an error]
131 - Division by zero: [value produced, or runtime error]
132
133 ### Scope and bindings
134
135 - Redeclaration in the same scope: [allowed or an error]
136 - Uninitialized variable holds: [value]
137 - Shadowing: [behavior]
138 - Undefined name: [static error with exit 65, or runtime error with exit 70]
139
140 ### Control flow and functions
141
142 - Logical operators return: [booleans, or the operand]
143 - Dangling else binds to: [which if]
144 - Closure capture of a loop variable: [per iteration, or shared]
145 - Function with no return statement produces: [value]
146 - Arity mismatch: [message and exit code]
147
148 ## Native functions
149
150 | Name | Arguments | Returns | Notes |
151 |---|---|---|---|
152 | [name] | [count and types] | [type] | [caveats] |
153
154 ## Errors and diagnostics
155
156 Message format:
157
158 ```
159 [one real static error]
160 [one real runtime error]
161 ```
162
163 | Failure | Exit code |
164 |---|---|
165 | [lexical error] | 65 |
166 | [syntax error] | 65 |
167 | [runtime error] | 70 |
168
169 ## Testing conventions
170
171 | Folder | Activity | Mode | Flag |
172 |---|---|---|---|
173 | tests/lab1 | Scanner | sidecar | `--tokenize` |
174 | tests/lab2 | Parser | sidecar | `--parse` |
175 | tests/lab3 | Evaluator | inline | `--eval` |
176 | tests/lab4 | Context | inline | none |
177 | tests/lab5 | Functions | inline | none |
178
179 ```
180 [specific tests]...
181 ```
182
183 Run locally with:
184
185 ```bash
186 curl -sSL https://raw.githubusercontent.com/WhiteLicorice/cmssc-124-harness/v1.1/run_tests.py
187 ↪ -o run_tests.py
188 ./build.sh
189 python3 run_tests.py tests/lab1
190 ```

```

```
191  ## Sample code
192
193  ```
194  [a short program]
195  ```
196
197  Output:
198
199  ```
200  [its output]
201  ```
202
203  ## Design rationale
204
205  [Why the language is the way it is. Cover the choices that surprised you, the
206  features you cut, and the decisions you reversed. Specific reasons, not
207  approval of your own work.]
208
209  ## Known limitations
210
211  - [What doesn't work, what is unimplemented, where behavior is worse than you
212    would like.]
213
214  ## Changelog
215
216  | Activity | What changed in the language |
217  |---|---|
218  | Lab 1 | [entry] |
```